

Brunel Hand API Specification

Arduino UNO R4 WiFi Control Server Reference

Version:	1.0.0
Date:	September 2026
Audience:	Engineers and Developers — Internal Use Only
Platform:	Arduino UNO R4 WiFi (RA4M1)
Protocol:	HTTP/1.1 over Wi-Fi, JSON payloads

This document defines the complete HTTP REST API for controlling the **Open Bionics Brunel Hand** via an **Arduino UNO R4 WiFi** HTTP server. It covers all endpoint categories — finger control, gesture execution, low-level actuator access, system diagnostics, safety protocols, intent-based controls, and optional sensor data — together with request/response formats, example JSON payloads, error codes, implementation notes, and a quick-start sketch outline. This specification is the authoritative reference for engineering teams integrating with the Brunel Hand over a local Wi-Fi network.

Safety Notice

This document describes hardware control of a powered prosthetic hand. All integrations must implement the Emergency Stop (`/safety/estop`) and Watchdog Keepalive (

/safety/keepalive

) endpoints. Never deploy without validating safety command response times on the target hardware.

Table of Contents

- 1.** Overview
- 2.** Endpoint Categories Summary
- 3.** Finger Control Endpoints — /finger
 - 3.1** Set Individual Finger Position
 - 3.2** Get Finger Positions
 - 3.3** Set Finger Speed (Global)
 - 3.4** Finger Range of Motion
- 4.** Hand Gesture Endpoints — /gesture
 - 4.1** Execute Predefined Gesture
 - 4.2** List Available Gestures
 - 4.3** Save Custom Gesture
 - 4.4** Delete Custom Gesture
- 5.** Actuator Commands — /actuator
 - 5.1** Set Actuator PWM
 - 5.2** Get Actuator State
 - 5.3** Actuator Calibration
- 6.** System Status Endpoints — /status
 - 6.1** System Health Check
 - 6.2** Battery Status
 - 6.3** Fault Log
 - 6.4** Reboot Device

- 7. Safety Commands — `/safety`
 - 7.1 Emergency Stop
 - 7.2 Release Emergency Stop
 - 7.3 Set Force Limit
 - 7.4 Get Safety Configuration
 - 7.5 Watchdog Keepalive
 - 8. Intent-Based Controls — `/intent`
 - 8.1 Execute Intent Command
 - 8.2 List Available Intents
 - 8.3 Intent to Gesture Mapping
 - 9. Optional Sensor Endpoints — `/sensor`
 - 9.1 EMG Signal Data
 - 9.2 Force / Pressure Sensor Data
 - 9.3 IMU / Orientation Data
 - 9.4 Tactile Sensor Array
 - 9.5 Sensor Configuration
 - 10. Error Codes Reference
 - 11. Arduino UNO R4 WiFi Implementation Notes
 - 12. Quick-Start Sketch Outline
-

1. Overview

The Brunel Hand API is a lightweight HTTP REST interface hosted directly on an **Arduino UNO R4 WiFi** microcontroller board. The Arduino acts as a Wi-Fi-connected HTTP server, accepting JSON-formatted commands from any client on the same local network and translating them into real-time actuator and motor control signals for the Brunel Hand prosthetic.

1.1 Connection Details

Property	Value	Notes
Base URL	http://<device-ip>/	Assign a static IP on the router for reliability
Protocol	HTTP/1.1	Over Wi-Fi (2.4 GHz). HTTPS not supported on UNO R4.
Default Port	80	Configurable in firmware at compile time
Data Format	JSON	Content-Type: application/json required on all POST/DELETE requests
Authentication	None	Local network only. Do not expose to public internet.
Max Request Body	512 bytes	SRAM constraint of RA4M1 chip
Max Response Body	512 bytes	Keep responses compact; omit optional fields when null

1.2 Standard Response Envelope

All API responses use a consistent JSON envelope regardless of endpoint:

```
{  "status":  "ok" | "error",    "message": "Human-readable status description",  "data":    { ... } }
```

- `status` — Always present. Either `"ok"` or `"error"`.
- `message` — Always present. A short plain-English description of the result or error.
- `data` — Present on success when there is a payload to return. Omitted or `null` on errors.

Note — Error Responses

On error, the HTTP status code will be non-2xx (see Section 10). The response

body will still follow the envelope format with

```
"status": "error"
```

, an

```
"error_code"
```

field, and a descriptive

```
"message"
```

. The

data

field will be absent.

```
{ "status": "error", "error_code": "INVALID_PARAM", "message": "Field  
'speed' must be in range 0–100." }
```

2. Endpoint Categories Summary

Category	Base Path	Methods	Description
Finger Control	/finger	GET, POST	Individual finger position, speed, and range-of-motion control
Hand Gestures	/gesture	GET, POST, DELETE	Predefined and custom named gesture commands
Actuator Commands	/actuator	GET, POST	Low-level PWM motor/servo actuator access and calibration

Category	Base Path	Methods	Description
System Status	/status	GET, POST	Device health, firmware info, battery, fault log, and reboot
Safety Commands	/safety	GET, POST	Emergency stop, fault reset, force limits, and watchdog keepalive
Intent-Based Controls	/intent	GET, POST	High-level semantic grip commands mapped automatically to finger positions
Sensor Endpoints	/sensor	GET, POST	EMG, force, IMU, and tactile sensor data (optional hardware required)

3. Finger Control Endpoints — /finger

These endpoints provide granular control over each of the five fingers of the Brunel Hand. Positions are expressed as a percentage (0–100%) of full range of motion, where **0%** is fully open and **100%** is fully closed. Speed values (0–100%) govern actuator velocity as a percentage of the configured maximum speed.

Finger Identifiers

Identifier	Description	Actuator
thumb	Thumb (opposition and flexion)	thumb_flexor, thumb_rotator
index	Index finger	index_flexor
middle	Middle finger	middle_flexor
ring	Ring finger (coupled with pinky)	ring_pinky_flexor
pinky	Little finger (coupled with	ring_pinky_flexor

Identifier	Description	Actuator
	ring)	

3.1 Set Individual Finger Position

POST /finger/position

Set the position (0–100%) and speed (0–100%) for one or more fingers simultaneously. Only the fingers included in the request body will be moved; omitted fingers retain their current positions.

Request Body:

```
{ "fingers": { "thumb": { "position": 75, "speed": 50 }, "index": { "position": 100, "speed": 80 }, "middle": { "position": 100, "speed": 80 }, "ring": { "position": 0, "speed": 50 }, "pinky": { "position": 0, "speed": 50 } } }
```

Request Fields:

Field

Type

Range

Required

Description

fingers

object

—

Yes

Map of finger identifiers to position/speed objects. At least one entry required.

fingers.*.position

integer

0–100

Yes

Target position as percentage of full range of motion.

fingers.*.speed

integer

0–100

No

Actuator speed as percentage of max. Defaults to global speed if omitted.

Response (200 OK):

```
{ "status": "ok", "message": "Finger positions updated", "data": { "applied":  
["thumb", "index", "middle", "ring", "pinky"] } }
```

3.2 Get Finger Positions

GET /finger/position

Returns the current target position of all five fingers as percentages.

Response (200 OK):

```
{ "status": "ok", "data": {  "thumb": 75,  "index": 100,  "middle": 100,  "ring": 0,  "pinky": 0  }}
```

Note

These values reflect the

commanded

target positions, not real-time encoder feedback. If optional position

sensors are installed, use

/sensor/force

or a future encoder endpoint for measured positions.

3.3 Set Finger Speed (Global)

POST /finger/speed

Sets a global default speed for all subsequent finger movements where no per-finger speed is specified. Does not immediately move any fingers.

Request Body:

```
{ "speed": 60 }
```

Request Fields:

Field

Type

Range

Required

Description

speed

integer

0–100

Yes

Global default speed percentage applied to all fingers.

Response (200 OK):

```
{ "status": "ok", "message": "Global speed set to 60%", "data": { "speed": 60 } }
```

3.4 Finger Range of Motion

GET /finger/range

Returns the calibrated minimum and maximum actuator limits for each finger. Values are hardware-specific and set during end-stop calibration.

Response (200 OK):

```
{ "status": "ok", "data": {  "thumb": { "min_pwm": 40, "max_pwm": 210,
"min_deg": 0, "max_deg": 90 },  "index": { "min_pwm": 35, "max_pwm": 220,
"min_deg": 0, "max_deg": 95 },  "middle": { "min_pwm": 38, "max_pwm": 215,
"min_deg": 0, "max_deg": 93 },  "ring": { "min_pwm": 36, "max_pwm": 218,
"min_deg": 0, "max_deg": 90 },  "pinky": { "min_pwm": 36, "max_pwm": 218,
"min_deg": 0, "max_deg": 85 }  } }
```

4. Hand Gesture Endpoints — /gesture

Gesture endpoints provide a higher-level abstraction over finger control. Predefined gestures are hard-coded in firmware as optimized finger position tables. Custom gestures are stored in EEPROM and survive power cycles. The gesture system is blocked while an ESTOP is active.

Predefined Gesture Reference

Gesture Name	Description	Typical Use Case
open	All fingers fully extended	Release object, resting position
close	All fingers fully flexed	Full fist, power grip
pinch	Thumb and index tip to tip, others relaxed	Picking small objects
tripod	Thumb, index, middle	Writing, utensils

Gesture Name	Description	Typical Use Case
	opposition	
power_grip	All fingers strongly closed, thumb opposed	Cylinder/bottle grasp
key_grip	Thumb lateral pinch against index side	Keys, cards, flat objects
point	Index extended, others curled	Pointing, button pressing
ok	Thumb and index circle, others extended	OK signal
relax	Natural resting curve, no tension	Idle / default state

4.1 Execute Predefined Gesture

POST /gesture/execute

Executes a named gesture at the specified speed. Works for both predefined firmware gestures and saved custom gestures.

Request Body:

```
{ "gesture": "pinch", "speed": 70 }
```

Field

Type

Required

Description

gesture

string

Yes

Name of the gesture to execute. Must match a predefined or saved custom gesture name.

speed

integer 0–100

No

Execution speed override. Defaults to current global speed.

Response (200 OK):

```
{ "status": "ok", "message": "Gesture 'pinch' executing", "data":  
  { "estimated_duration_ms": 450 } }
```

4.2 List Available Gestures

GET /gesture/list

Returns all available gesture names, including both predefined firmware gestures and any saved custom gestures.

Response (200 OK):

```
{ "status": "ok", "data": { "predefined":  
["open","close","pinch","tripod","power_grip","key_grip","point","ok","relax"],  
"custom": ["my_grip", "lab_test_1"] } }
```

4.3 Save Custom Gesture

POST /gesture/save

Captures the current finger positions and saves them as a named custom gesture in EEPROM. The gesture can then be executed by name via `/gesture/execute`.

Request Body:

```
{ "name": "my_grip", "overwrite": false }
```

Field

Type

Required

Description

name

string

Yes

Unique name for the gesture. Alphanumeric and underscores only. Max 24 characters.

overwrite

boolean

No

If

true

, overwrites an existing gesture with the same name. Defaults to

false

. Returns an error if

false

and name exists.

Response (200 OK):

```
{ "status": "ok", "message": "Gesture 'my_grip' saved to EEPROM", "data":  
{ "name": "my_grip", "snapshot": { "thumb": 75, "index": 100, "middle": 100,  
"ring": 0, "pinky": 0 } } }
```

4.4 Delete Custom Gesture

DELETE /gesture/{name}

Permanently deletes a previously saved custom gesture from EEPROM.
Predefined firmware gestures cannot be deleted.

Parameter

Location

Type

Description

{name}

URL path

string

Name of the custom gesture to delete (e.g.,

/gesture/my_grip

).

Response (200 OK):

```
{ "status": "ok", "message": "Gesture 'my_grip' deleted" }
```

5. Actuator Commands — /actuator

Actuator endpoints expose low-level, direct motor control. These are intended for advanced use cases such as calibration, testing, and firmware development.

Incorrect PWM values can damage the hand mechanism or cause

unintended motion. Use high-level finger or gesture endpoints for normal operation.

Warning — Advanced Use Only

Raw actuator commands bypass the position management and safety clamping layers. Always ensure the hand is unloaded and has clear range of motion before issuing raw PWM commands. Force limits set via `/safety/force_limit` remain active.

Actuator Identifiers

Actuator ID	Controls	PWM Range
thumb_flexor	Thumb open/close flexion	0-255
thumb_rotator	Thumb opposition rotation	0-255
index_flexor	Index finger flexion	0-255
middle_flexor	Middle finger flexion	0-255
ring_pinky_flexor	Ring and pinky coupled flexion	0-255

5.1 Set Actuator PWM

POST /actuator/pwm

Sends a raw PWM value to a named actuator for a specified duration. The actuator will return to idle (PWM 0) when the duration expires.

Request Body:

```
{ "actuator": "thumb_flexor", "pwm": 180, "duration_ms": 500 }
```

Field

Type

Range

Required

Description

actuator

string

See table

Yes

Actuator identifier string.

pwm

integer

0–255

Yes

Raw PWM duty cycle value.

duration_ms

integer

1–5000

No

Duration in milliseconds to hold the PWM. Defaults to 200 ms if omitted.

Response (200 OK):

```
{ "status": "ok", "message": "PWM applied", "data": {  "actuator":  
"thumb_flexor",  "pwm":      180,  "duration_ms": 500  }}
```

5.2 Get Actuator State

GET /actuator/state

Returns the current PWM value, direction flag, and fault status for all actuators.

Response (200 OK):

```
{ "status": "ok", "data": {  "thumb_flexor":  { "pwm": 0, "direction": "idle",  
"fault": false },  "thumb_rotator":  { "pwm": 0, "direction": "idle",  "fault": false },  
"index_flexor":  { "pwm": 180, "direction": "forward", "fault": false },  
"middle_flexor":  { "pwm": 180, "direction": "forward", "fault": false },  
"ring_pinky_flexor":{ "pwm": 0, "direction": "idle",  "fault": false }  }}
```

5.3 Actuator Calibration

POST /actuator/calibrate

Runs an automatic end-stop calibration sequence on the specified actuator. The sequence drives the actuator to its physical limits and records the PWM boundaries. Results are stored in EEPROM and used by all high-level endpoints.

Ensure the hand has full, unobstructed range of motion before running calibration.

Request Body:

```
{ "actuator": "index_flexor" }
```

Response (200 OK):

```
{ "status": "ok", "message": "Calibration complete for index_flexor", "data":  
  { "actuator": "index_flexor", "min_pwm": 35, "max_pwm": 220,  
    "duration_ms": 3200  } }
```

6. System Status Endpoints — /status

Status endpoints provide diagnostic and device health information. These endpoints are read-only except for the reboot command.

6.1 System Health Check

GET /status/health

Returns overall system health, firmware version, uptime in seconds, Wi-Fi signal strength (RSSI), processor temperature, and CPU load estimate.

Response (200 OK):

```
{ "status": "ok", "data": {   "firmware":    "1.0.0",   "uptime_s":    3600,
  "wifi_rssi_dbm": -62,   "temperature_c": 34.2,   "cpu_load_pct": 12,
  "estop_active": false,   "watchdog_ok":  true  }}
```

Field

Type

Description

firmware

string

Firmware version string (semver)

uptime_s

integer

Seconds since last boot

wifi_rssi_dbm

integer

Wi-Fi signal strength in dBm. Values above -70 dBm recommended.

temperature_c

float

RA4M1 chip temperature (Celsius). Alert threshold: 70°C.

cpu_load_pct

integer

Estimated processor utilization percentage

estop_active

boolean

Whether ESTOP is currently engaged

watchdog_ok

boolean

Whether the watchdog timer is healthy

6.2 Battery Status

GET /status/battery

Returns battery voltage, charge percentage, charge state, and low-battery alert flag.

Response (200 OK):

```
{ "status": "ok", "data": { "voltage_v": 7.4, "percent": 82,
"charging": false, "low_battery_alert": false } }
```

Important

When

low_battery_alert

is

true

(typically below 10%), the firmware will automatically reduce maximum speed and force limits. At critical level (

<

5%), an automatic ESTOP is triggered.

6.3 Fault Log

GET /status/faults

Returns the list of recent fault events with timestamps (seconds since boot) and fault codes. The log retains up to the last 20 entries in RAM; it is cleared on reboot.

Response (200 OK):

```
{ "status": "ok", "data": { "fault_count": 2, "faults": [ { "uptime_s": 120,
"code": "ACTUATOR_FAULT", "actuator": "index_flexor", "detail": "Stall detected" },
{ "uptime_s": 305, "code": "WATCHDOG_TIMEOUT", "detail": "Keepalive missed" }
] }}
```

6.4 Reboot Device

POST /status/reboot

Gracefully reboots the Arduino UNO R4 WiFi controller. Issues an ESTOP first, waits 500 ms, then triggers a software reset. Requires explicit confirmation in the request body to prevent accidental reboots.

Request Body:

```
{ "confirm": true }
```

Response (200 OK):

```
{ "status": "ok", "message": "Rebooting in 500ms. Reconnect after 5 seconds." }
```

Warning

The HTTP connection will drop immediately after the 500 ms delay. The device will be unreachable for approximately 5–10 seconds while it reconnects to Wi-Fi.

7. Safety Commands — /safety

Safety endpoints have the **highest execution priority** in the firmware. The ESTOP command is processed before any queued motion commands. All motion-generating endpoints (/finger, /gesture, /actuator, /intent) check ESTOP state at the start of request handling and return 403 ESTOP_ACTIVE if engaged.

7.1 Emergency Stop

POST /safety/estop

Immediately halts all actuators by setting all PWM outputs to zero. Sets the global ESTOP flag. No request body fields are required or processed — the command is executed the moment the URI is matched. Should be implemented using a hardware interrupt pin for minimum latency.

Request Body:

```
{}
```

Response (200 OK):

```
{ "status": "ok", "message": "ESTOP engaged. All actuators halted." }
```

Implementation Requirement

Wire a dedicated GPIO pin as a hardware interrupt tied directly to the ESTOP handler function. This ensures ESTOP can trigger even if the main loop is blocked by a long-running operation. Network-only ESTOP is insufficient for safety-critical deployments.

7.2 Release Emergency Stop

POST /safety/estop/release

Clears the ESTOP state and re-enables motion commands. Requires explicit operator confirmation. Fault log should be reviewed before releasing ESTOP.

Request Body:

```
{ "confirm": true }
```

Response (200 OK):

```
{ "status": "ok", "message": "ESTOP released. Motion commands re-enabled." }
```

7.3 Set Force Limit

POST /safety/force_limit

Sets the maximum allowable total grip force across all fingers in Newtons. The firmware monitors optional force sensors and triggers an ESTOP if this limit is exceeded. When no force sensor hardware is present, this limit is enforced via PWM current limiting as a best-effort approximation.

Request Body:

```
{ "max_force_n": 15.0 }
```

Field

Type

Range

Required

Description

max_force_n

float

1.0–30.0

Yes

Maximum total fingertip grip force in Newtons. Recommended default: 15.0 N.

Response (200 OK):

```
{ "status": "ok", "message": "Force limit set", "data": { "max_force_n": 15.0 } }
```

7.4 Get Safety Configuration

GET /safety/config

Returns the full current safety configuration including ESTOP state, force limit, compliance mode, and watchdog timeout.

Response (200 OK):

```
{ "status": "ok", "data": {  "estop_active":    false,  "max_force_n":    15.0,
  "compliance_mode":  "soft",  "watchdog_timeout_ms":2000,
  "low_battery_estop": true,  "temp_estop_c":    70  }}
```

Field

Description

compliance_mode

"soft"

— actuators back-drive under external force;

"rigid"

— actuators hold position against force.

watchdog_timeout_ms

Milliseconds before an absent keepalive triggers auto-ESTOP.

low_battery_estop

Whether critically low battery triggers automatic ESTOP.

temp_estop_c

Chip temperature (°C) threshold above which ESTOP is auto-triggered.

7.5 Watchdog Keepalive

POST /safety/keepalive

Resets the watchdog timer. The controlling application must call this endpoint at an interval shorter than `watchdog_timeout_ms` (default: 2000 ms). If the keepalive is not received in time — indicating a crashed or disconnected client — the firmware will automatically engage ESTOP. This ensures the hand cannot remain in motion if the controller loses connectivity.

Request Body:

```
{}
```

Response (200 OK):

```
{ "status": "ok", "message": "Watchdog reset", "data": { "next_deadline_ms": 2000 } }
```

Implementation Requirement

Implement keepalive sending on a dedicated timer thread or interval in your client. Do not rely on motion command timing to implicitly reset the watchdog. The keepalive endpoint should be called at least every 1000 ms when the default 2000 ms timeout is configured.

8. Intent-Based Controls — /intent

Intent endpoints provide the highest level of abstraction in the API. Instead of specifying finger positions or gesture names, the client describes *what the hand should do* semantically. The firmware maps each intent to the optimal finger

configuration, taking into account object size and force parameters. This layer is ideal for integrations with AI inference, EMG classifiers, or voice control systems.

Available Intents

Intent Name	Description	Resolved Gesture	Supports Object Size
grasp_cylinder	Wrap grip around a cylindrical object	power_grip	Yes
grasp_sphere	Spherical object grasp	power_grip (modified)	Yes
grasp_flat	Grasp a flat object (card, plate)	key_grip	Yes
pinch_small	Fine pinch for very small objects	pinch	No
pinch_large	Three-finger pinch for larger objects	tripod	No
writing_grip	Pen/pencil holding configuration	tripod	No
phone_grip	Hold a smartphone or flat device	key_grip (modified)	No
tool_grip	Grip a handle or tool shaft	power_grip	Yes
wave_hello	Repetitive open/close wave animation	Animated sequence	No
thumbs_up	Thumbs up gesture	Custom	No

8.1 Execute Intent Command

POST /intent/execute

Executes a high-level grip intent. The firmware automatically resolves the intent to the appropriate gesture and finger position targets based on the provided parameters.

Request Body:

```
{ "intent": "grasp_cylinder", "object_size": "medium", "force_pct": 60 }
```

Field

Type

Required

Description

intent

string

Yes

Intent name from the available intents table.

object_size

string

No

Size hint:

"small"

,

"medium"

, or

"large"

. Used to adjust finger aperture. Ignored for intents that do not support sizing.

force_pct

integer 0–100

No

Grip force as percentage of the configured

max_force_n

. Defaults to 50%.

Response (200 OK):

```
{ "status": "ok", "message": "Intent 'grasp_cylinder' executing", "data":  
{ "resolved_gesture": "power_grip", "finger_targets": { "thumb": 80,  
"index": 90, "middle": 90, "ring": 85, "pinky": 80 },  
"estimated_duration_ms": 520 }}
```

8.2 List Available Intents

GET /intent/list

Returns all available intent names with human-readable descriptions and whether they support the `object_size` parameter.

Response (200 OK):

```
{ "status": "ok", "data": { "intents": [ { "name": "grasp_cylinder",
"description": "Cylindrical object power grip", "supports_size": true }, { "name":
"pinch_small", "description": "Fine fingertip pinch", "supports_size": false },
{ "name": "wave_hello", "description": "Wave animation sequence",
"supports_size": false } ] }}
```

8.3 Intent to Gesture Mapping

GET /intent/mapping

Returns the complete lookup table mapping each intent name to its resolved gesture name and base finger position targets (before object-size adjustment). Useful for debugging intent resolution.

Response (200 OK):

```
{ "status": "ok", "data": { "mapping": { "grasp_cylinder": { "gesture":
"power_grip", "base_targets": { "thumb": 80, "index": 90, "middle": 90, "ring":
85, "pinky": 80 } }, "pinch_small": { "gesture": "pinch",
"base_targets": { "thumb": 70, "index": 80, "middle": 0, "ring": 0, "pinky": 0 } }
}}
```

9. Optional Sensor Endpoints — /sensor

Optional Hardware Required

All sensor endpoints require optional sensor hardware to be physically installed and enabled in firmware configuration. Requests to disabled sensors return

500 SENSOR_UNAVAILABLE

. Check firmware build flags to enable only the sensors present on your hardware revision.

9.1 EMG Signal Data

GET /sensor/emg

Returns the most recent raw and normalized EMG (electromyography) channel readings from surface EMG electrodes. Normalized values are scaled 0.0–1.0 against the calibrated baseline and peak values for each channel.

Response (200 OK):

```
{ "status": "ok", "data": { "channels": [ { "id": 0, "raw": 412, "normalized": 0.42, "label": "flexor" }, { "id": 1, "raw": 198, "normalized": 0.19, "label": "extensor" } ], "sample_rate_hz": 200, "timestamp_ms": 45213 } }
```

9.2 Force / Pressure Sensor Data

GET /sensor/force

Returns the most recent fingertip pressure sensor readings in Newtons for each finger, plus the total summed force. Requires FSR or load-cell sensors at each fingertip.

Response (200 OK):

```
{ "status": "ok", "data": {  "thumb":  2.1,  "index":  3.4,  "middle":  3.0,
  "ring":   0.0,  "pinky":  0.0,  "total_n": 8.5  }}
```

9.3 IMU / Orientation Data

GET /sensor/imu

Returns accelerometer (m/s²), gyroscope (rad/s), and computed orientation (degrees) from an onboard IMU module (e.g., LSM6DSO or equivalent).

Orientation uses a simple complementary filter; true sensor fusion requires off-board processing.

Response (200 OK):

```
{ "status": "ok", "data": {  "accel":    { "x":  0.02, "y": -9.81, "z":  0.11 },  "gyro":
  { "x":  0.003, "y": -0.001, "z":  0.002 },  "orientation": { "roll": -1.2, "pitch": 0.4, "yaw":
  92.1 }  }}
```

9.4 Tactile Sensor Array

GET /sensor/tactile

Returns a 2D grid of pressure values from a fingertip tactile array sensor (e.g., a 4×4 taxel array). Each value is 0–255 representing contact pressure at that taxel location. Only available when a compatible tactile array is installed on the specified finger.

Response (200 OK):

```
{ "status": "ok", "data": { "finger": "index", "grid_rows": 4, "grid_cols": 4,
"values": [ [0, 12, 45, 30], [0, 8, 92, 85], [0, 5, 88, 77], [0, 0, 21, 10]
] }}
```

9.5 Sensor Configuration

POST /sensor/config

Enables or disables individual sensor modules and sets their sample rates. Changes take effect immediately but are not persisted across reboots unless saved to EEPROM via a separate configuration save endpoint.

Request Body:

```
{ "emg": { "enabled": true, "sample_rate_hz": 200 }, "imu": { "enabled": false },
"force": { "enabled": true }, "tactile": { "enabled": false } }
```

Response (200 OK):

```
{ "status": "ok", "message": "Sensor configuration updated", "data": { "emg":
{ "enabled": true, "sample_rate_hz": 200 }, "imu": { "enabled": false },
"force": { "enabled": true }, "tactile": { "enabled": false } } }
```

10. Error Codes Reference

All error responses include a non-2xx HTTP status code, a JSON body with "status": "error", an "error_code" string, and a human-readable "message". The following table defines all possible error codes in this API.

HTTP Status	Error Code	Meaning	Resolution
200	—	Success	No action required
400	INVALID_PARAM	Missing, null, or out-of-range parameter in request body	Check field types and valid ranges; re-read request body spec
400	UNKNOWN_GESTURE	Gesture name not recognized (predefined or custom)	Query GET /gesture/list for valid names
400	UNKNOWN_INTENT	Intent name not recognized	Query GET /intent/list for valid names
400	UNKNOWN_ACTUATOR	Actuator identifier not recognized	Use only the five defined actuator IDs in Section 5
400	GESTURE_NAME_EXISTS	Custom gesture name already exists and overwrite is false	Set "overwrite": true or choose a different name
400	PREDEFINED_GESTURE	Attempt to delete or overwrite a predefined firmware gesture	Custom gestures only; predefined gestures cannot be removed
403	ESTOP_ACTIVE	Motion command blocked — Emergency Stop is currently engaged	Call POST /safety/estop/release with "confirm": true
409	MOTION_IN_PROGRESS	A motion command is already executing; concurrent motion not supported	Wait for current motion to complete or issue ESTOP, then retry
500	ACTUATOR_FAULT	Hardware actuator fault detected (stall, over-current, or	Check GET /status/faults; inspect wiring; may

HTTP Status	Error Code	Meaning	Resolution
		driver error)	require recalibration
500	SENSOR_UNAVAILABLE	Requested sensor is not physically present or is disabled in firmware	Check hardware and firmware sensor enable flags; use POST /sensor/config
500	EEPROM_WRITE_FAIL	Failed to write custom gesture or config to EEPROM	EEPROM may be worn or full; check EEPROM write cycle count
503	WATCHDOG_TIMEOUT	Watchdog timer expired — device has entered safe mode and engaged ESTOP	Restore keepalive cadence; release ESTOP; verify client connectivity

11. Arduino UNO R4 WiFi Implementation Notes

11.1 Required Libraries

Library	Version	Purpose
WiFiS3	Bundled with UNO R4 board package	Wi-Fi connectivity and HTTP server on RA4M1
ArduinoJson	v6.21+ (v7 compatible)	JSON parsing and serialization. Install via Library Manager.
Servo / PWM	Bundled	PWM output for actuator motor drivers
EEPROM	Bundled	Persistent storage of custom gestures and calibration data
WDT_RA	Bundled with UNO R4 board package	RA4M1 hardware watchdog timer peripheral access

11.2 HTTP Server and Routing

- Use `WiFiServer server(80);` to initialize the HTTP server on port 80.
- In `loop()`, call `WiFiClient client = server.available();` and read the HTTP request line into a `String`.
- Implement endpoint routing using `request.startsWith("/gesture/execute")` or equivalent `String` matching on the first line of the HTTP request.
- Always read and discard all HTTP headers before reading the JSON body to avoid buffer starvation.
- Limit request line and header buffering to 256 bytes; JSON body buffer to 256–512 bytes.

11.3 JSON Parsing with ArduinoJson

- Use `StaticJsonDocument<256>` for small request bodies to avoid heap fragmentation.
- Always check `deserializeJson()` return value; return `400 INVALID_PARAM` on parse failure.
- For response serialization, use `StaticJsonDocument<256>` and `serializeJson(doc, client)` to write directly to the network client.
- Avoid `String` concatenation for JSON building — it causes heap fragmentation on constrained SRAM.

11.4 Memory Constraints

- The RA4M1 on the Arduino UNO R4 WiFi provides **32 KB SRAM**. Budget memory carefully.
- Keep all request and response buffers at or below **512 bytes each**.
- Pre-allocate static buffers at startup rather than using dynamic allocation inside request handlers.
- Store gesture tables and calibration data in `PROGMEM` or EEPROM, not RAM.

- Limit the fault log to the last 10–20 entries; store in a fixed-size circular buffer in RAM.

11.5 Watchdog Implementation

- Use the RA4M1 WDT peripheral via the `WDT` register set or the `WDT_RA` Arduino library.
- Configure the WDT timeout to match `watchdog_timeout_ms` (default 2000 ms).
- Do **not** reset the WDT inside `loop()` unconditionally — reset it only when a valid keepalive HTTP request is received at `/safety/keepalive`.
- When the WDT fires, the RA4M1 triggers a hardware reset. Implement a startup flag in EEPROM to detect watchdog-caused resets and report them in the fault log on next boot.

11.6 Emergency Stop Hardware Wiring

- Dedicate a GPIO pin (e.g., D2) as a hardware interrupt input tied to a physical ESTOP button or safety relay output.
- Use `attachInterrupt(digitalPinToInterrupt(2), estopISR, FALLING)`; where `estopISR` immediately drives all PWM outputs to zero.
- The ISR must be minimal: set a global volatile `estop_active` flag and zero all PWM registers. Do not call WiFi or JSON functions from an ISR.
- The software `POST /safety/estop` endpoint must also set the same `estop_active` flag through the same code path.

11.7 Concurrency and Blocking Calls

- The Arduino UNO R4 runs a **single-threaded** event loop. Never use `delay()` inside HTTP request handlers — it blocks all networking and safety checks.
- Implement motion timing using non-blocking `millis()`-based state machines.
- Long operations (calibration sequences) should return an immediate `202 Accepted` response and execute the sequence asynchronously on subsequent `loop()` iterations.

11.8 CORS Headers for Browser Dashboards

If the API is consumed by a browser-based monitoring dashboard hosted on a different origin, include the following HTTP response headers on all responses:

```
Access-Control-Allow-Origin: * Access-Control-Allow-Methods: GET, POST, DELETE, OPTIONS Access-Control-Allow-Headers: Content-Type
```

Also handle the HTTP `OPTIONS` preflight request method by responding immediately with these headers and a 204 status, with no body. Note that CORS headers have no security value on a local network; they are required only for browser fetch compatibility.

12. Quick-Start Sketch Outline

The following pseudocode sketch outlines a minimal Arduino UNO R4 WiFi HTTP server that connects to Wi-Fi, initializes the server, and handles a `POST /gesture/execute` request with ArduinoJson parsing. This is a structural template — fill in motor control calls, safety logic, and additional routes as required.

```
// _____ // Brunel Hand
API Server – Quick-Start Sketch Outline // Arduino UNO R4 WiFi | Firmware
v1.0.0 // _____ #include
<WiFiS3.h> #include <ArduinoJson.h> // — Configuration
const char* WIFI_SSID =
"YOUR_SSID"; const char* WIFI_PASSWORD = "YOUR_PASSWORD"; const int
SERVER_PORT = 80; // — Globals

WiFiServer
server(SERVER_PORT); volatile bool estop_active = false; unsigned long
last_keepalive_ms = 0; const unsigned long WATCHDOG_TIMEOUT_MS = 2000; // —
ESTOP Hardware Interrupt Service Routine void estopISR() {
estop_active = true; // Zero all PWM outputs immediately
analogWrite(THUMB_PIN, 0); analogWrite(INDEX_PIN, 0);
analogWrite(MIDDLE_PIN, 0); analogWrite(RING_PIN, 0); } // — setup()
void setup()
{ Serial.begin(115200); // Attach hardware ESTOP interrupt (active-low
button on D2) pinMode(2, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(2), estopISR, FALLING); // Connect
to Wi-Fi Serial.print("Connecting to Wi-Fi..."); WiFi.begin(WIFI_SSID,
WIFI_PASSWORD); while (WiFi.status() != WL_CONNECTED) { delay(500);
Serial.print("."); } Serial.println("\nConnected. IP: " +
WiFi.localIP().toString()); // Start HTTP server server.begin();
Serial.println("HTTP server started on port " + String(SERVER_PORT)); } //
— loop() void loop()
{ // — Watchdog check (software fallback) if (!
estop_active) { if (millis() - last_keepalive_ms > WATCHDOG_TIMEOUT_MS) {
```

```

estop_active = true;      analogWrite(THUMB_PIN, 0);
analogWrite(INDEX_PIN, 0);      analogWrite(MIDDLE_PIN, 0);
analogWrite(RING_PIN, 0);      // Log WATCHDOG_TIMEOUT
fault      }      }      // — Serve HTTP request
                                WiFiClient client = server.available();
if (!client) return;      // Read request line (e.g. "POST /gesture/execute
HTTP/1.1")      String requestLine = client.readStringUntil('\n');
requestLine.trim();      // Discard remaining headers until blank line      while
(client.available()) {      String headerLine = client.readStringUntil('\n');
headerLine.trim();      if (headerLine.length() == 0) break;      }      // —
Route: POST /gesture/execute      if
(requestLine.startsWith("POST /gesture/execute")) {      // Check ESTOP
before any motion command      if (estop_active)
{      sendErrorResponse(client, 403, "ESTOP_ACTIVE", "ESTOP is engaged.");
return;      }      // Read JSON body (max 256 bytes)
StaticJsonDocument<256> reqDoc;      String body = client.readString();
DeserializationError err = deserializeJson(reqDoc, body);      if (err) {
sendErrorResponse(client, 400, "INVALID_PARAM", "JSON parse failed.");
return;      }      const char* gesture = reqDoc["gesture"];      int speed
= reqDoc["speed"] | 60;      // default 60%      if (!gesture)
{      sendErrorResponse(client, 400, "INVALID_PARAM", "Field 'gesture'
required.");      return;      }      // Execute gesture (expand this with
your gesture table lookup)      bool success = executeGesture(String(gesture),
speed);      if (!success) {      sendErrorResponse(client, 400,
"UNKNOWN_GESTURE", "Gesture not found.");      return;      }      // Build
and send success response      StaticJsonDocument<256> resDoc;
resDoc["status"] = "ok";      resDoc["message"] = String("Gesture '") +
gesture + "' executing";      resDoc["data"]["estimated_duration_ms"] = 450;
client.println("HTTP/1.1 200 OK");      client.println("Content-Type:
application/json");      client.println("Access-Control-Allow-Origin: *");
client.println("Connection: close");      client.println();
serializeJson(resDoc, client);      }      // — Route: POST /safety/keepalive
                                else if (requestLine.startsWith("POST
/safety/keepalive")) {      last_keepalive_ms = millis();      // Reset watchdog
timer      sendSimpleOk(client, "Watchdog reset");      }      // — Route: POST
/safety/estop      else if
(requestLine.startsWith("POST /safety/estop")) {      estop_active = true;
analogWrite(THUMB_PIN, 0);      // ... zero all actuators      sendSimpleOk(client,
"ESTOP engaged. All actuators halted.");      }      // — Fallback: 404 Not Found
                                else {      sendErrorResponse(client, 404,
"NOT_FOUND", "Endpoint not found.");      }      client.stop();      }      // — Helper:
Send error JSON response      void
sendErrorResponse(WiFiClient&c, int code,      const
char* errCode, const char* msg) {      StaticJsonDocument<128> doc;
doc["status"] = "error";      doc["error_code"] = errCode;      doc["message"]
= msg;      c.println("HTTP/1.1 " + String(code) + " Error");
c.println("Content-Type: application/json");      c.println("Connection:
close");      c.println();      serializeJson(doc, c);      c.stop();      }      // — Helper:
Send simple OK response      void
sendSimpleOk(WiFiClient&c, const char* msg) {      StaticJsonDocument<96> doc;
doc["status"] = "ok";      doc["message"] = msg;      c.println("HTTP/1.1 200
OK");      c.println("Content-Type: application/json");      c.println("Connection:
close");      c.println();      serializeJson(doc, c);      c.stop();      }

```

Next Steps

Expand the route handler block with all endpoints defined in this specification.

Implement

`executeGesture()`

with a lookup table mapping gesture names to finger position arrays. Add ArduinoJson parsing for all request body fields. Integrate motor driver PWM output using your specific hardware driver circuit.